

Desafio final - Aprendizaje por Refuerzo II

Integrantes:

[a2221] Cesar Octavio Mejia

Docente:

Esp. Ing. Miguel Augusto Azar

Repositorio del proyecto:

<https://github.com/comejia/ceia-rl-ii-project>

Introducción

Este trabajo presenta la solución del desafío final de la materia Aprendizaje por Refuerzo II, de la Maestría en Inteligencia Artificial de la FIUBA. Este desafío consiste en la implementación de algunas de las técnicas vistas durante el cursado.

Para ese trabajo se eligió el algoritmo Proximal Policy Optimization (PPO) aplicado al control de un brazo robótico de 7 grados de libertad y con el objetivo de alcanzar un punto final, para este caso una pelota ubicada en una posición aleatoria.

Algoritmo PPO (Proximal Policy Optimization)

PPO es un algoritmo de aprendizaje por refuerzo basado en políticas (*policy gradient methods*) desarrollado para mejorar la estabilidad del entrenamiento.

Los métodos clásicos de Policy Gradient actualizan la política utilizando directamente el gradiente de la recompensa:

$$\nabla_{\theta} J(\theta)$$

Sin embargo, estas actualizaciones pueden producir cambios demasiado grandes en la política, provocando inestabilidad durante el aprendizaje. PPO introduce una restricción que evita modificar demasiado la política en cada actualización.

PPO utiliza una arquitectura Actor-Critic, formada por dos redes neuronales:

- Actor

Aprende la política:

$$\pi_{\theta}(a | s)$$

Su función es decidir qué acción tomar dado un estado.

- Critic

Aprende la función de valor:

$$V_{\phi}(s)$$

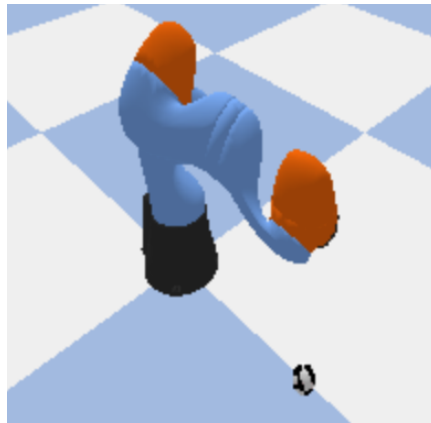
que estima la recompensa futura esperada desde un estado determinado.

Entorno utilizado

Para este trabajo se desarrolló un entorno personalizado utilizando la interfaz de Gymnasium, permitiendo integrar el problema de control dentro del framework estándar de Reinforcement Learning.

El entorno representa un sistema robótico donde el agente debe controlar la posición del efector final (End Effector) para alcanzar una posición objetivo (Target).

Para la simulación de la física del robot a controlar se utilizó **PyBullet**, en particular el brazo KUKA, de 7 grados de libertad, que ya se encuentra integrado en la librería.



PyBullet es de uso completamente gratuito y open source, lo que lo convierte en una alternativa frente a MuJoCo, además de ser más simple tanto en la instalación como en la configuración, ideal para un prototipo. Algunas de sus ventajas son:

- Integración directa con Python sin dependencias complejas.
- Incluye un **servidor GUI interactivo** para visualización en tiempo real sin configuración adicional.
- Permite simulación tanto en modo gráfico como *headless* (sin interfaz), útil para entrenamiento de varias épocas.
- Activa comunidad en entornos de RL, con integración directa a OpenAI Gym / Gymnasium.

Implementación

En esta sección se muestran las partes del código más relevantes del desarrollo realizado.

Al ser un entorno generado se tuvo que definir varias cosas, entre ellas se destacan las más importantes:

- Número de articulaciones

Se utilizaron los 7 grados de libertad del brazo robótico, que es el valor por defecto.

- Espacio de acciones

Son acciones continuas y define qué puede hacer el agente. En particular se definió que el agente puede mover cada articulación como máximo ± 0.05 rad.

- Espacio de observación:

El espacio de observación está compuesto por las posiciones y velocidades articulares, la posición del efector final (End Effector), la posición del objetivo y el error cartesiano entre ambos.

- Máximo de pasos

Para este trabajo se definieron 200 pasos, como máximo, para llegar al objetivo en cada episodio.

```
# 7 articulaciones
self.n_joints = 7

# acciones continuas
self.action_space = spaces.Box(
    low=-0.05,
    high=0.05,
    shape=(self.n_joints,),
    dtype=np.float32
)

# observaciones
obs_size = 23

self.observation_space = spaces.Box(
    low=-np.inf,
    high=np.inf,
    shape=(obs_size,),
    dtype=np.float32
)

self.max_steps = 200
```

- Estado terminal

Se definió como estado terminal cuando la distancia al objetivo es menor a un valor dado, en este caso cuando la distancia es menor a 5 cm del target.

```
terminated = distance < 0.05

if terminated:
    reward += 100
```

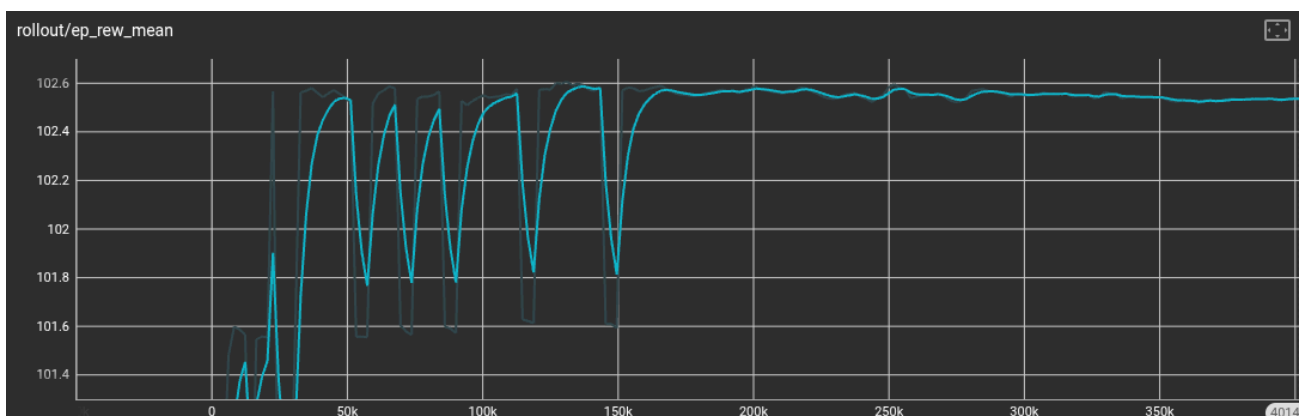
- Función de recompensas

Inicialmente se evaluó una recompensa basada únicamente en la distancia absoluta al objetivo ($R=-d$). Sin embargo, esta estrategia no proporcionó resultados buenos durante el entrenamiento. Posteriormente se adoptó una recompensa basada en progreso, calculada como la reducción de la distancia al objetivo entre dos pasos consecutivos. Y finalmente, se incorporó una técnica de reward shaping que otorga una recompensa adicional cuando el efector final se encuentra a menos de 20 cm del objetivo.

```
def _compute_reward_progress_shaping(self):  
  
    current_distance = self._distance_to_target()  
  
    progress = self.previous_distance - current_distance  
  
    # bonus suave cuando está cerca  
    if current_distance < 0.2:  
        progress += 1.0 * (0.2 - current_distance)  
  
    self.previous_distance = current_distance  
  
    return progress
```

Resultados y Conclusiones

A continuación se muestra el gráfico de convergencia durante el entrenamiento:



La curva de recompensa media por episodio muestra una fase inicial de exploración seguida por una mejora abrupta del desempeño alrededor de los 30.000 pasos de entrenamiento. Posteriormente se observan fluctuaciones transitorias características de PPO, asociadas al proceso de actualización de la política. A partir de aproximadamente 160.000 pasos, la recompensa media se estabiliza en torno a 102.5, indicando que el agente alcanza consistentemente el objetivo y que la política ha convergido. La ausencia de

mejoras significativas después de este punto sugiere que el entrenamiento ha alcanzado un comportamiento estable y cercano al óptimo para la tarea definida.

Luego al evaluar el sistema, se obtuvo las siguientes métricas:

```
=====
Success rate: 100.00%      M 38 # total_timesteps=200_000,
Average reward: 102.582    U 39 # tb_log_name="rew_distance_progress_shaping_
Average final distance: 0.045 U 40 # )
Average steps to success: 65.050 41
Best distance: 0.039        U 42 print("Entrenamiento finalizado")
=====                    43
                             44 model.save("ppo_robot_arm_v2_5cm")
                             45
```

Los resultados de evaluación muestran que la política aprendida resuelve exitosamente la tarea de reaching en el 100% de los episodios evaluados. La distancia final promedio al objetivo fue de 0.045 m, por debajo del umbral de éxito de 0.05 m. Además, el agente alcanzó el objetivo en un promedio de 65 pasos, muy por debajo del máximo permitido de 200 pasos por episodio. La recompensa promedio obtenida (102.582) coincide con los valores observados durante el entrenamiento, indicando una convergencia estable y una adecuada capacidad de generalización de la política aprendida. En conjunto, estos resultados evidencian que PPO logró aprender una estrategia efectiva y consistente para controlar el brazo robótico y alcanzar la posición objetivo.

Como observación final se destaca que si bien estas métricas evidencian que el agente fue capaz de resolver exitosamente la tarea en la totalidad de los episodios evaluados, definir la función de recompensa iterativamente fue crucial para llegar a una convergencia estable.